

1.Explain the difference between a has-a and is-a relationship among classes

An is-a relationship in object-oriented programming refers to inheritance, where one class is a specialized type of another class. For example, a Truck is a Vehicle, meaning the Truck class inherits the properties and methods of the Vehicle class and fits into a hierarchical structure. In contrast, a has-a relationship refers to composition, where one class contains another class as part of its structure. For instance, a Car has an Engine, meaning the Car class includes an Engine object rather than inheriting from it. The key difference is that is-a describes a type relationship, while has-a describes a usage or ownership relationship between classes.

2. If a base class has a public method go() and a derived class has a public method stop() Which methods will be available to an object of the derived class?

An object of the derived class will have access to both the go() method from the base class and the stop() method defined in the derived class. This is because inheritance allows the derived class to use all public methods of the base class, in addition to its own methods.

3. Compare and contrast implementing an abstract class to an interface.

Implementing an abstract class and an interface both allow you to define a common structure for classes, but they are used in different ways. An abstract class can contain both abstract methods (without implementation) and concrete methods (with implementation), as well as instance variables, and a class can only extend one abstract class. In contrast, an interface typically only defines method signatures (and constants), without providing implementation, and a class can implement multiple interfaces. Abstract classes are best used when classes share common behavior and state, while interfaces are used to define a contract that different classes can follow, even if they are not closely related.

4. Compare and contrast an abstract class to an interface?

An abstract class and an interface are both used to define common behavior for other classes, but they differ in flexibility and purpose. An abstract class can include both abstract methods and fully implemented methods, as well as instance variables to store state, and a class can only extend one abstract class. In contrast, an interface mainly defines method signatures (and constants) without implementation, and a class can implement multiple interfaces at the same time. Abstract classes are typically used when related classes need to share code and state, while interfaces are used to define a common contract that can be applied to different, unrelated classes.

5. Use the following classes to answer the questions below

```
Interface Wo {  
Public int doThat();  
}
```

```
Public class Bo {  
Private int x;
```

```
Public Bo (int z) {  
x=z  
}
```

```
Public int doThis() {  
return(2);  
}
```

```
Public int doNow() {  
return(15);  
}  
}
```

```
Public class Roo extends Bo implements Wo {
```

```
Public Roo(  
super(1);  
}
```

```
Public into doThis() {  
Return(10);  
}
```

```
Private int doThat() {  
return(20);  
}  
}
```

a) What type of method is doThat() in Wo?

The doThat() method in Wo is an abstract method. This is because all methods declared in an interface (like Wo) are abstract by default, meaning they only provide a method signature and no implementation. Classes that implement the interface are required to provide the actual implementation of this method.

b) What is Wo

Wo is an interface. It defines a contract that any class (like Roo) must follow. In this case, Wo declares the method doThat(), which means any class that implements Wo is supposed to provide an implementation for that method.

c) Why is doThat() implemented in Roo

doThat() is implemented in Roo because Roo implements the Wo interface. When a class implements an interface, it is required to provide an implementation for all of the interface's abstract methods. Since Wo declares doThat(), Roo must define that method to fulfill the contract of the interface.

d) List the methods available to Roo object

A Roo object will have access to methods from both its superclass Bo and its own class.

From Bo, it inherits:

doThis()

doNow()

doThat()